

Python pour le trading — Document 7/8

La qualité, le test et l'outillage

1. Les tests unitaires avec pytest

Un **test unitaire** vérifie automatiquement qu'un petit morceau de code se comporte comme prévu. Plutôt que de tester à la main après chaque modification, on écrit des tests qui s'exécutent en une commande. pytest est l'outil standard en Python.

```
# Le code à tester (dans ordres.py)
def valeur_notionnelle(quantite, prix):
    return quantite * prix

# Le test (dans test_ordres.py)
def test_valeur_notionnelle():
    assert valeur_notionnelle(5, 5234.50) == 26172.50    # cas normal
    assert valeur_notionnelle(0, 5234.50) == 0           # cas limite

def test_valeur_avec_quantite_negative():
    assert valeur_notionnelle(-2, 100) == -200           # comportement attendu
```

Un test est une fonction dont le nom commence par `test_`. À l'intérieur, `assert` vérifie qu'une condition est vraie : si elle l'est, le test passe ; sinon, il échoue et pytest le signale. On lance tous les tests d'un coup avec la commande `pytest`. L'intérêt : après chaque modification, on relance les tests et on sait immédiatement si quelque chose est cassé. On teste le cas normal, mais aussi les **cas limites** (zéro, valeurs négatives), là où se cachent les bugs.

1.1 Paramétrer un test

Tester plusieurs jeux de valeurs en copiant-collant des `assert` devient vite répétitif.

`@pytest.mark.parametrize` exécute la même fonction de test sur une liste de cas, et signale lesquels échouent, un par un.

```
import pytest
from ordres import valeur_notionnelle

@pytest.mark.parametrize("quantite, prix, attendu", [
    (5, 5234.50, 26172.50),    # cas normal
    (0, 5234.50, 0),           # quantité nulle
    (-2, 100, -200),           # quantité négative
])
def test_valeur_notionnelle(quantite, prix, attendu):
    assert valeur_notionnelle(quantite, prix) == attendu
```

Chaque tuple devient un test distinct. pytest affiche trois résultats, pas un seul : si le cas négatif casse, on le voit aussitôt, sans que les autres soient masqués. C'est la façon idiomatique de couvrir de nombreux cas sans dupliquer le code.

1.2 Les fixtures : préparer le terrain

Beaucoup de tests ont besoin de la même préparation : un portefeuille initial, une connexion simulée, un jeu de données. Une **fixture** est une fonction qui fournit cet objet, et que pytest injecte automatiquement dans chaque test qui la réclame.

```
import pytest

@pytest.fixture
def portefeuille():
    # préparation commune, recrée pour chaque test
    return {"ES": 10, "NQ": -5}

def test_position_longue(portefeuille):      # pytest injecte la fixture
    assert portefeuille["ES"] > 0

def test_position_courte(portefeuille):
    assert portefeuille["NQ"] < 0
```

Il suffit de nommer la fixture en paramètre du test pour la recevoir. pytest la recrée pour chaque test, ce qui garantit qu'ils n'interfèrent pas entre eux. Les fixtures évitent la duplication et clarifient ce que chaque test suppose au départ.

1.3 Mesurer la couverture

Avoir des tests ne dit pas combien de code ils exercent réellement. La **couverture** (coverage) mesure le pourcentage de lignes exécutées par la suite de tests, via le greffon `pytest-cov`.

```
# pip install pytest-cov
# pytest --cov=ordres
#
# Name          Stmts  Miss  Cover
# -----
# ordres.py      42      6    86%
```

Une couverture de 86 % signifie que 14 % des lignes ne sont jamais exécutées par les tests — souvent des cas d'erreur oubliés. La couverture n'est pas une fin en soi (du code couvert peut rester mal testé), mais elle révèle les angles morts. On vise une bonne couverture des chemins critiques, pas nécessairement 100 %.

Pourquoi ça compte. Sur un système de trading, un bug introduit par une modification peut être très coûteux. Une batterie de tests détecte instantanément une régression avant qu'elle n'atteigne la production. C'est un filet de sécurité que tout employeur sérieux attend.

2. Le mocking : tester sans dépendances réelles

Comment tester une fonction qui appelle une API de courtier, sans réellement passer d'ordre ni dépendre du réseau ? Avec un **mock** : un faux objet qui imite le vrai, et qu'on contrôle. Cela rend les tests rapides, fiables et sans effet de bord.

```
from unittest.mock import Mock

def envoyer_ordre(courtier, ordre):
    reponse = courtier.passer(ordre)      # appelle le vrai courtier
    return reponse["statut"]

def test_envoyer_ordre():
```

```

faux_courtier = Mock()                # un faux courtier
faux_courtier.passer.return_value = {"statut": "execute"} # réponse simulée

resultat = envoyer_ordre(faux_courtier, {"symbole": "ES"})
assert resultat == "execute"
faux_courtier.passer.assert_called_once() # vérifie qu'on l'a bien appelé

```

Le Mock remplace le vrai courtier par un faux qu'on programme : ici, on lui dit de renvoyer {"statut": "execute"} quand on appelle passer. Le test vérifie alors le comportement de envoyer_ordre sans toucher à un vrai courtier — aucun ordre réel, aucune dépendance au réseau. On peut même vérifier que la fonction a bien appelé le courtier, avec assert_called_once. C'est indispensable pour tester du code qui interagit avec l'extérieur.

***Pourquoi ça compte.** On ne peut pas passer de vrais ordres pour tester. Le mocking permet de vérifier toute la logique de trading sans risque ni dépendance réseau, ce qui rend les tests reproductibles et sûrs — une compétence attendue dans tout code financier.*

3. Le logging : garder une trace

Le **logging** (journalisation) consiste à enregistrer ce que fait le programme, avec des niveaux d'importance. C'est radicalement supérieur aux print : on peut filtrer par gravité, horodater, et écrire dans un fichier. Dans un système temps réel, c'est souvent le seul moyen de comprendre après coup ce qui s'est passé.

```

import logging

logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger(__name__)

log.debug("Détail technique, ignoré en production")
log.info("Ordre ES envoyé, quantité 5")
log.warning("Latence élevée détectée")
log.error("Échec de connexion au courtier")
log.critical("Perte de tous les flux de marché")

```

Le logging propose des **niveaux** de gravité croissante : DEBUG (détails), INFO (déroulement normal), WARNING (anomalie non bloquante), ERROR (échec), CRITICAL (urgence). En réglant le niveau (ici INFO), on choisit ce qu'on garde : en production, on ignore les DEBUG pour ne pas noyer l'information. Chaque message est horodaté automatiquement. Contrairement aux print, on peut rediriger ces journaux vers un fichier et les analyser plus tard.

3.1 Handlers et formatters : écrire où l'on veut

basicConfig suffit pour démarrer, mais un vrai système choisit où vont les messages et sous quelle forme. Deux objets s'en chargent : un **handler** décide de la destination (console, fichier, réseau), un **formatter** décide de la mise en forme de chaque ligne.

```

import logging

log = logging.getLogger("trading")

```

```

log.setLevel(logging.DEBUG)

# Un handler vers un fichier, un autre vers la console
fichier = logging.FileHandler("trading.log")
fichier.setLevel(logging.INFO)      # le fichier garde INFO et au-dessus

console = logging.StreamHandler()
console.setLevel(logging.WARNING)   # la console n'affiche que les alertes

# Un formatter commun : date, niveau, nom, message
forme = logging.Formatter("%(asctime)s  %(levelname)-8s  %(name)s  %(message)s")
fichier.setFormatter(forme)
console.setFormatter(forme)

log.addHandler(fichier)
log.addHandler(console)

```

Ici, tout (à partir d'INFO) est écrit dans `trading.log` pour l'analyse a posteriori, tandis que la console ne montre que les WARNING et plus, pour ne pas noyer l'opérateur. Le formatter compose chaque ligne à partir d'attributs comme `%(asctime)s` (l'horodatage), `%(levelname)s` (le niveau) ou `%(message)s` (le texte). Un même logger peut ainsi alimenter plusieurs sorties, chacune avec son propre seuil.

3.2 Journaliser une exception

Quand une erreur survient, on veut garder non seulement le message mais aussi la trace complète (la pile d'appels). Dans un bloc `except`, `log.exception` le fait automatiquement.

```

try:
    envoyer_ordre(courtier, ordre)
except Exception:
    # enregistre le message ET la trace complète de l'erreur
    log.exception("Échec de l'envoi de l'ordre %s", ordre)

```

`log.exception` équivaut à `log.error(..., exc_info=True)` : il consigne le message et la pile d'appels, ce qui permet de retrouver l'origine exacte du problème. Les `%s` dans le message sont remplis par les arguments suivants — on évite ainsi de construire la chaîne quand le niveau est désactivé. C'est la différence entre « ça a planté » et « voici exactement où et pourquoi ».

Pourquoi ça compte. Quand un système tourne en continu et qu'un problème survient à 3h du matin, les journaux sont la seule façon de reconstituer ce qui s'est passé. Un logging soigné, par niveaux, est vital pour surveiller et diagnostiquer un système de trading — exactement la surveillance demandée dans l'offre.

4. Le débogage

Quand un bug résiste, le **débogueur** permet d'arrêter le programme à un endroit précis et d'inspecter l'état des variables, pas à pas. Python intègre `pdb`.

```

def calculer_pnl(ordres):
    total = 0
    for ordre in ordres:
        breakpoint()      # le programme s'arrête ICI
        total += ordre.valeur # on peut inspecter 'ordre' et 'total'
    return total

```

La fonction `breakpoint()` met le programme en pause à cet endroit et ouvre une invite où l'on peut afficher les variables (taper le nom d'une variable), exécuter **ligne par ligne**, et comprendre exactement ce qui se passe. C'est bien plus puissant que de parsemer le code de `print` : on explore l'état réel du programme au moment du bug. Les commandes courantes sont `n` (ligne suivante), `c` (continuer), `p` variable (afficher).

***Pourquoi ça compte.** Un débogueur fait gagner un temps considérable face à un bug subtil : au lieu de deviner, on observe directement l'état du programme. Sur du code de trading complexe, c'est un outil de diagnostic essentiel.*

5. Le profilage de performance

Quand un programme est trop lent, le **profilage** identifie **où** il passe son temps, au lieu de deviner. Inutile d'optimiser au hasard : on mesure d'abord. Python fournit `cProfile` et `timeit`.

```
import cProfile

# Profiler : où le temps est-il dépensé ?
cProfile.run("analyser_historique(donnees)")
# -> liste les fonctions par temps cumulé : on voit le goulot d'étranglement

# timeit : mesurer précisément un petit bout de code
import timeit
duree = timeit.timeit("sum(range(1000))", number=10000)
```

`cProfile.run` exécute le code et rapporte, fonction par fonction, le temps passé : on repère immédiatement le **goulot d'étranglement**, c'est-à-dire la partie lente qui mérite d'être optimisée. `timeit` mesure précisément un petit fragment en le répétant plusieurs fois. La règle : **mesurer avant d'optimiser**. Souvent, 90 % du temps est passé dans 10 % du code ; le profilage révèle ces 10 %, pour concentrer l'effort là où il compte.

Q8b. Dans une sortie `cProfile`, que signifient `tottime` et `cumtime` ?

Réponse. `tottime` est le temps passé dans la fonction seule (hors sous-appels) ; `cumtime` inclut les sous-appels. Trier par `cumtime` fait remonter le goulot d'étranglement.

5.1 Lire une sortie `cProfile`

Prenons une vraie sortie sur une fonction de trading lente : une moyenne mobile recalculée naïvement à chaque pas sur 20 000 prix.

```
# analyse.py
def moyenne(fenetre):
    return sum(fenetre) / len(fenetre)

def moyennes_mobiles(prix, n):
    res = []
    for i in range(n, len(prix)):
        res.append(moyenne(prix[i-n:i])) # resomme toute la fenêtre à chaque pas
    return res

def analyser_ticks(prix):
```

```

    return moyennes_mobiles(prix, 50)
import cProfile
cProfile.run("analyser_ticks(prix)", sort="cumulative")
79804 function calls in 0.036 seconds

```

Ordered by: cumulative time

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	0.036	0.036	analyse.py:11(analyser_ticks)
1	0.013	0.013	0.036	0.036	analyse.py:5(moyennes_mobiles)
19950	0.007	0.000	0.020	0.000	analyse.py:2(moyenne)
19950	0.010	0.000	0.010	0.000	{built-in method builtins.sum}
19950	0.003	0.000	0.003	0.000	{method 'append' of 'list' objects}
19951	0.002	0.000	0.002	0.000	{built-in method builtins.len}

On lit la table par colonnes. `ncalls` est le nombre d'appels, `totttime` le temps passé dans la fonction elle-même (hors sous-appels), `cumtime` le temps cumulé (sous-appels inclus). Triée par `cumtime`, la table montre que tout le temps part dans `moyennes_mobiles`, et précisément dans les 19 950 appels à `sum` : c'est le goulot. La leçon : resommer toute la fenêtre à chaque pas est gaspilleur — une somme glissante, ou la fonction `rolling` de Pandas (document 5), supprimerait ce coût. Le profilage a pointé l'endroit exact à corriger.

Pourquoi ça compte. Optimiser à l'aveugle fait perdre du temps et complique le code inutilement. Le profilage indique précisément quoi accélérer — crucial quand la latence compte, comme dans un système de trading. C'est le prélude au document 8 sur la performance.

6. Style et structure du code

Un code de production doit être **lisible et cohérent**, car il est relu et modifié par plusieurs personnes. Deux conventions comptent : la norme de style **PEP 8**, et l'organisation en modules.

```

# Un "linter" comme ruff signale automatiquement les écarts de style
#   ruff check mon_projet/

# Organisation en modules (fichiers) et packages (dossiers)
#   ordres.py          -> la logique des ordres
#   marche.py          -> la connexion au marché
#   test_ordres.py     -> les tests

```

La **PEP 8** est la convention de style officielle de Python (nommage, espacement, longueur des lignes). Un **linter** comme `ruff` la vérifie automatiquement et signale les écarts. Organiser le code en **modules** (fichiers thématiques) et **packages** (dossiers) le rend navigable : chaque responsabilité a sa place. Un code propre et bien rangé est plus facile à comprendre, à tester et à faire évoluer — ce que tout employeur valorise.

Pourquoi ça compte. Dans une équipe, du code lisible et cohérent réduit les erreurs et accélère le travail de tous. Respecter **PEP 8** et structurer son projet signale un développeur professionnel, au-delà du simple « ça marche ».

Récapitulatif

1. pytest : des tests automatiques (fonctions `test_` avec `assert`) ; couvrir les cas normaux et limites.

2. Mocking : remplacer une dépendance réelle (courtier, API) par un faux contrôlé, pour tester sans risque.

3. Logging : journaliser par niveaux (DEBUG à CRITICAL) plutôt que des print ; indispensable au suivi temps réel.

4. Débogage : breakpoint() pour arrêter et inspecter l'état du programme pas à pas.

5. Profilage : cProfile/timeit pour mesurer où le temps est passé ; mesurer avant d'optimiser.

6. Style : PEP 8 et linters (ruff), organisation en modules et packages.

Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

Sur les tests et le mocking

Q1. Qu'est-ce qu'un test unitaire et comment l'écrit-on avec pytest ?

Réponse. Une vérification automatique qu'un morceau de code se comporte comme prévu. Avec pytest, on écrit une fonction dont le nom commence par test_ et on y place des assert qui vérifient des conditions.

Q2. Pourquoi tester aussi les cas limites ?

Réponse. Parce que c'est là que se cachent souvent les bugs : valeurs nulles, négatives, vides, extrêmes. Tester seulement le cas normal laisse passer ces erreurs.

Q2b. À quoi servent @pytest.mark.parametrize et les fixtures ?

Réponse. parametrize exécute le même test sur plusieurs jeux de valeurs (chacun signalé séparément). Une fixture fournit une préparation commune (objet, données), injectée automatiquement dans les tests qui la nomment.

Q3. Qu'est-ce qu'un mock et pourquoi l'utiliser ?

Réponse. Un faux objet qui imite un vrai (courtier, API) et qu'on contrôle. Il permet de tester la logique sans dépendre du réseau ni provoquer d'effet réel (comme passer un vrai ordre).

Sur le logging et le débogage

Q4. Pourquoi préférer le logging aux print ?

Réponse. Le logging propose des niveaux de gravité, horodate les messages, peut écrire dans un fichier et filtrer ce qu'on garde. Les print n'offrent rien de tout cela et sont inadaptés à un système en production.

Q5. Cite les niveaux de logging du moins au plus grave.

Réponse. DEBUG, INFO, WARNING, ERROR, CRITICAL.

Q5b. À quoi servent un handler et un formatter ?

Réponse. Un handler choisit la destination des messages (fichier, console...), un formatter leur mise en forme (horodatage, niveau, texte). Un même logger peut combiner plusieurs handlers avec des seuils différents.

Q6. Qu'apporte un débogueur (breakpoint) par rapport aux print ?

Réponse. Il arrête le programme à un endroit précis et permet d'inspecter l'état réel des variables, d'avancer ligne par ligne. On observe au lieu de deviner.

Sur le profilage et le style

Q7. Que fait le profilage et quelle est sa règle d'or ?

Réponse. Il identifie où le programme passe son temps (le goulot d'étranglement). Règle d'or : mesurer avant d'optimiser, plutôt que d'optimiser au hasard.

Q8. Quelle est la différence entre cProfile et timeit ?

Réponse. cProfile analyse tout un programme et rapporte le temps par fonction. timeit mesure précisément un petit fragment de code en le répétant plusieurs fois.

Q9. Qu'est-ce que la PEP 8 et un linter ?

Réponse. La PEP 8 est la convention de style officielle de Python (nommage, espacement...). Un linter comme ruff vérifie automatiquement le respect de ces règles et signale les écarts.

Document suivant (8/8) : la performance — pourquoi Python peut être lent, la vectorisation comme solution, le multiprocessing pour le calcul, et les accélérateurs numba/Cython.